

Integrando o Routine v1 ao Supabase

Guia passo a passo: projeto, banco de dados, autenticação, segurança (RLS) e teste de ponta a ponta.

O que você vai fazer	Tempo estimado
Criar o projeto Supabase e rodar as migrations	5 min
Configurar a autenticação por email e senha	2 min
Conectar o app (variáveis de ambiente) e testar	5 min
Validar a segurança (RLS) e o isolamento entre usuários	5 min

Pré-requisitos: Node.js 20+ instalado, o repositório `routinev1` clonado com as dependências instaladas (`npm install`) e uma conta no Supabase (gratuita, em supabase.com).

1. Como a integração funciona

O app não possui backend próprio. O Supabase fornece três peças que o Routine v1 usa diretamente:

- **PostgreSQL**: guarda perfis, pilares, hábitos, agendas, registros diários e tarefas.
- **Auth**: cadastro e login por email e senha; a sessão fica em cookies.
- **Row Level Security (RLS)**: regras no banco que garantem que cada usuário só enxergue e altere os próprios dados.

Arquivo do projeto	Papel na integração
supabase/migrations/0001_init.sql	Cria tabelas, índices, triggers e a função que gera o perfil no cadastro.
supabase/migrations/0002_rls.sql	Liga o RLS e cria as políticas de acesso por usuário.
lib/supabase/client.ts	Cliente do navegador (Client Components).
lib/supabase/server.ts	Cliente do servidor (Server Components e Server Actions), lê/grava cookies.
lib/supabase/proxy.ts + proxy.ts	Renova a sessão a cada requisição e redireciona quem não está logado para /login.
lib/auth.ts	requireUser(): revalida o usuário dentro de cada Server Action.
lib/actions/*.ts	Mutações (criar hábito, marcar progresso, criar tarefa...) via Server Actions.
lib/data/*.ts	Leituras usadas pelas páginas (hábitos, logs, tarefas).

O Next.js 16 renomeou `middleware.ts` para `proxy.ts`. Este projeto já usa o novo nome.

2. Criar o projeto no Supabase

Passo 1 Criar conta e projeto

- Acesse **supabase.com** e entre (email ou GitHub).
- Clique em **New project**.
- Nome: `routinev1`. Defina uma **senha do banco** e guarde-a em um gerenciador de senhas.
- Região: escolha a mais próxima de você (ex.: South America - São Paulo).
- Clique em **Create new project** e aguarde cerca de 2 minutos.

A senha do banco **não** vai para o app. Ela só é necessária para conexões diretas ao Postgres (por exemplo, a Supabase CLI). O app usa apenas a URL e a chave pública.

3. Criar as tabelas e as regras de segurança

Você vai executar dois arquivos SQL, **nesta ordem**. A forma mais simples é pelo SQL Editor do painel.

Passo 2 Rodar 0001_init.sql

- No painel do projeto, abra **SQL Editor** e clique em **New query**.
- Abra o arquivo `supabase/migrations/0001_init.sql` no seu editor, copie todo o conteúdo e cole na query.

- Clique em **Run**. O resultado esperado é *Success. No rows returned*.

Passo 3 Rodar 0002_rls.sql

- Crie outra query, cole o conteúdo de 0002_rls.sql e execute.

Passo 4 Conferir o resultado

- Vá em **Table Editor**: devem existir as tabelas `profiles`, `pillars`, `habits`, `habit_schedules`, `habit_logs` e `tasks`.
- Cada tabela deve mostrar o selo **RLS enabled**.

O que cada migration cria

Item	Descrição
Tabelas	6 tabelas normalizadas; hábitos nunca são apagados com histórico (arquivamento via campo <code>active</code>).
unique (<code>habit_id</code> , <code>log_date</code>)	Garante um único registro por hábito por dia.
habit_schedules temporal	Mudar a frequência fecha a linha antiga (<code>end_date</code>) e cria outra, preservando o histórico.
Trigger <code>handle_new_user</code>	Cria automaticamente a linha em <code>profiles</code> quando alguém se cadastra.
Trigger <code>enforce_habit_log_ownership</code>	Impede gravar log em hábito de outro usuário, mesmo que o cliente envie um <code>user_id</code> forjado.
Políticas RLS	Todas as tabelas: o usuário só acessa linhas cujo dono é <code>auth.uid()</code> .

4. Configurar a autenticação

Passo 5 Confirmar o provedor de email

- Vá em **Authentication > Sign In / Providers** (ou **Providers**).
- Verifique que **Email** está habilitado.

Passo 6 Desativar a confirmação de email (apenas em desenvolvimento)

Por padrão o Supabase exige que o usuário clique em um link enviado por email antes de entrar. Como o app redireciona para `/today` logo após o cadastro, a confirmação ativada faz o usuário voltar para `/login` sem sessão.

- Em **Authentication > Sign In / Providers > Email**, desligue **Confirm email** e salve.

Em produção, mantenha a confirmação de email ligada e configure um provedor SMTP próprio (o envio padrão do Supabase tem limite baixo de emails por hora). Nesse caso, ajuste o fluxo de cadastro para mostrar uma tela "verifique seu email".

Passo 7 Configurar URLs (necessário ao publicar)

- Vá em **Authentication > URL Configuration**.
- **Site URL**: em desenvolvimento, `http://localhost:3000`; em produção, o domínio do app.
- **Redirect URLs**: inclua as mesmas origens. Isso será exigido quando você ativar login com Google/Apple.

5. Conectar o app ao projeto

Passo 8 Copiar as credenciais

- No painel, abra **Project Settings > API** (ou o botão **Connect** no topo do projeto).
- Copie a **Project URL** (formato `https://xxxx.supabase.co`).
- Copie a chave pública: **anon public** ou, nos painéis mais novos, a **Publishable key**. As duas funcionam com o app.

Passo 9 Criar o arquivo .env.local

Na raiz do projeto, crie (ou edite) o arquivo `.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=https://SEU-PROJETO.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=sua-chave-publica-aqui
```

Nunca coloque a chave `service_role` (ou `secret key`) no app nem em variáveis com prefixo `NEXT_PUBLIC_`. Ela ignora o RLS e dá acesso total ao banco. O arquivo `.env.local` já está no `.gitignore`; não o envie ao GitHub.

Passo 10 Iniciar o app

```
npm install
npm run dev
```

Acesse `http://localhost:3000`. Sem sessão, você será redirecionado para `/login`. Se alterar o `.env.local` com o servidor ligado, reinicie o `npm run dev`.

6. Testar de ponta a ponta

Passo 11 Criar a primeira conta

- Em `/signup`, informe nome, email e senha (mínimo de 6 caracteres).
- Você deve cair em `/today` com o estado vazio "Você ainda não possui hábitos para hoje".
- No painel, em **Authentication > Users**, o usuário aparece; em **Table Editor > profiles** existe a linha correspondente com o `timezone` detectado pelo navegador.

Passo 12 Percorrer o fluxo principal

- **+ > Pilar**: crie "Saúde" com ícone e cor.
- **+ > Hábito**: crie "Água" (tipo Fazer, Quantidade, meta 3000 ml, todos os dias).
- **+ > Hábito**: crie "Sem Instagram" (tipo Evitar).
- Em **Hoje**: toque nos hábitos, use `+250 ml` / `+500 ml` e confira o contador e a porcentagem.
- **+ > Tarefa**: crie uma tarefa para hoje e marque como concluída.
- Abra **Semana** e **Progresso** e confirme que os números refletem o que você marcou.
- No **Table Editor > habit_logs**, confira que `log_date` é o dia local (não o dia em UTC).

7. Validar a segurança (RLS)

O objetivo é provar que o isolamento entre usuários vem do **banco**, não do frontend.

Passo 13 Conferir as políticas

No SQL Editor, execute:

```
select schemaname, tablename, policyname, cmd
from pg_policies
where schemaname = 'public'
order by tablename;
```

Devem aparecer políticas para as seis tabelas. Para checar se o RLS está ligado:

```
select relname, relrowsecurity
from pg_class
where relnamespace = 'public'::regnamespace and relkind = 'r';
```

A coluna `relrowsecurity` deve ser **true** para todas.

Passo 14 Teste de isolamento com dois usuários

- Crie uma segunda conta (janela anônima do navegador) e cadastre um hábito nela.
- Volte à primeira conta: o hábito da segunda **não** deve aparecer em nenhuma tela.
- Tentar abrir diretamente `/habits/ID-DO-OUTRO-USUARIO` deve resultar em 404, porque o banco não devolve a linha.

O botão **Run** do SQL Editor executa com privilégios de administrador e por isso ignora o RLS. Para simular um usuário logado, use o app, não o SQL Editor.

8. Problemas comuns

Sintoma	Causa provável e solução
Fica preso na tela de login após o cadastro	"Confirm email" está ligado. Desligue em Authentication > Sign In / Providers > Email (passo 6) ou confirme o email recebido.
Erro "Invalid API key" ou "Failed to fetch"	URL ou chave errada no <code>.env.local</code> , ou servidor não reiniciado depois de editar o arquivo.
Erro "relation public.profiles does not exist"	As migrations não foram executadas, ou foram em ordem inversa. Rode 0001 e depois 0002.
Usuário criado, mas sem linha em profiles	A trigger <code>on_auth_user_created</code> não existe (0001 incompleta). O app tem uma rede de segurança que cria o perfil no primeiro acesso, mas rode o 0001 completo.
Listas vazias mesmo após criar dados	RLS ligado sem políticas (0002 não rodou) bloqueia tudo. Reexecute o 0002.
Erro "new row violates row-level security policy"	O <code>user_id</code> enviado não corresponde ao usuário logado, ou a sessão expirou. Faça login novamente.
"Email rate limit exceeded"	Limite de envio de emails do plano gratuito. Desligue a confirmação em dev ou configure SMTP próprio.
Registro do dia aparece no dia errado	Confira <code>profiles.timezone</code> . Ele deve ser o IANA do usuário (ex.: <code>America/Sao_Paulo</code>).

9. Próximos passos

Passo 15 Gerar os tipos TypeScript a partir do banco (recomendado)

Hoje o arquivo `types/database.types.ts` foi escrito à mão. Com o banco criado, gere o arquivo oficial:

```
npx supabase login
npx supabase gen types typescript --project-id SEU_PROJECT_ID > types/database.types.ts
```

O Project ID aparece em Project Settings > General. Depois de gerar, rode `npx tsc --noEmit` para garantir que nada quebrou.

Passo 16 Publicar (ex.: Vercel)

- Importe o repositório na Vercel.
- Em **Settings > Environment Variables**, adicione `NEXT_PUBLIC_SUPABASE_URL` e `NEXT_PUBLIC_SUPABASE_ANON_KEY`.
- No Supabase, atualize **Site URL** e **Redirect URLs** com o domínio publicado (passo 7).

Passo 17 Login com Google ou Apple (Fase 3)

- Em **Authentication > Providers**, habilite o provedor e informe o Client ID/Secret criados no console do Google ou da Apple.
- Nenhuma alteração de schema é necessária: o perfil continua sendo criado pela mesma trigger.

Checklist final: projeto criado; 0001 e 0002 executados; RLS ativo nas 6 tabelas; email provider habilitado; `.env.local` preenchido; cadastro e login funcionando; segundo usuário não enxerga dados do primeiro.